

Programación Orientada a Objetos

Trabajo Práctico Integrador

UNQ-Shop

Integrantes: Britos Bautista, Tolazzi Coral, Enzo Perez

Emails:

Bautista Britos: bautista.britos05@gmail.com

Enzo Perez: tarealauty@gmail.com

Coral Tolazzi: CoralTolazzi@gmail.com

Decisiones de Diseño

A la hora de decidir cómo se dividirán los productos, nos topamos con que estos debían contar con un stock y un número de ventas, cosa que nos llevó a agregar un objeto Catálogo que contenga a estos Producto mediante un HashMap. Y desde ahí un Pedido que se comunique con el Catalogo para poder hacer las compras.

En el punto 2.3 Para los métodos de envío decidimos que cada uno calcule su costo y sus días de entrega a su manera, ya que varían según el criterio de cada envío. Creamos una interfaz MetodoDeEnvio con los métodos calcularCosto y estimarDias, y cada envío la implementa distinto: el estándar se apoya en la librería CorreoArgentina que da el enunciado y estima entre 5 y 7 días, el express calcula un porcentaje sobre el total del pedido y entrega en 1 día, y el retiro en sucursal tiene costo cero. Así el pedido no necesita saber qué tipo de envío es: solo le pide el costo y los días al método que tenga asignado.

En el punto 2.4 para los métodos de pago decidimos que los cuatro pasos del pago (validar, reservar, ejecutar y notificar) vayan siempre en el mismo orden, ya que lo único que cambia es cómo los hace cada medio. Dejamos pagar() como final para fijar ese orden, los tres primeros pasos abstractos para que cada medio los implemente a su manera, y notificarResultado con una implementación por defecto que las subclases pisan solo si lo necesitan. También creamos una interfaz-API por cada medio (TarjetaApi, TransferenciaApi y BilleteraApi) que se pasa por constructor, como pide el enunciado. En Transferencia, reservarFondos no hace nada porque la transferencia es directa.

En cuanto a los tipos de datos que utilizamos, decidimos para los números utilizar double, debido a que tiene más funciones que los demás tipos y nos resultó la mejor opción. Para todo lo que sea texto utilizamos Strings, al igual que para retornar los exportables del punto 2.8.

En el punto 2.6, cuando realizamos el UML detectamos que había que implementar el patrón Strategy, pero a la hora de ir al código, nos topamos con una dificultad para los criterios complejos. Por lo cual, luego de debatir, dimos con que había que utilizar un Composite además del Strategy. Costó ver, pero una vez debatido, la implementación fue fácil, ya que es tal cual muestra el libro de Gamma. Luego el Strategy de este punto y la implementación del patrón Visitor en el punto 2.8 fue fácil, sin modificaciones en cuanto a las estructuras vistas. No tuvimos grises más que eso, ni ningún caso borde.

Patrones de Diseño Utilizados

2.1 Catálogo De Productos

Patrón: Composite.

Justificación: Para la resolución de este punto utilizamos el patrón **Composite**.

Encapsulamos la familia de algoritmos en Producto. A su vez, al tener Paquetes que unen Producto o utilizan varios, nos encontramos con la necesidad de hacer una composición mediante el patrón Composite.

Roles de las clases en Composite:

- Component: Producto
- Leaf : ProductoIndividual

- Composite: Paquete

2.2 Ciclo De Vida

Patrón: State.

Justificación: Para la resolución de este punto utilizamos el patrón **State**. Con la evolución constante que transita un Pedido, nos encontramos con la necesidad de hacer una composición mediante el patrón State.

Roles de las clases en State:

- Context: Pedido
- State: Estado
- Concrete State: Borrador, Confirmado, En_Preparacion, Enviado, Entregado y Cancelado

2.3 Métodos de envío

Patrón: Strategy.

Justificación: Para la resolución de este punto utilizamos el patrón Strategy. Cada método de envío calcula su costo y estima sus días con un criterio propio (peso y distancia, porcentaje sobre el total, o retiro sin cargo), pero todos cumplen el mismo contrato. Encapsulamos cada algoritmo en su propia clase y los hacemos intercambiables, de modo que el Pedido delega el cálculo en la estrategia que tenga asignada sin conocer su implementación concreta ni necesitar condicionales para elegir el comportamiento.

Roles de las clases en Strategy:

- Strategy: MetodoDeEnvio
- Concrete Strategy: EnvioEstandar, EnvioExpress y RetiroEnSucursal
- Context: Pedido

2.4 Métodos de pago

Patrón: Template Method.

Justificación: Para la resolución de este punto utilizamos el patrón Template Method. El procesamiento de un pago sigue siempre la misma secuencia de pasos en el mismo orden (validar datos, reservar fondos, ejecutar transacción y notificar resultado), pero cada medio de pago implementa esos pasos de manera diferente. Definimos ese esqueleto invariante en un único método plantilla de la superclase y delegamos cada paso variable en las subclases. El paso de notificación tiene una implementación por defecto en la superclase que las subclases pueden redefinir sin estar obligadas a hacerlo, mientras que los pasos de validación, reserva y ejecución son obligatorios y específicos de cada medio.

Roles de las clases en Template Method:

- AbstractClass: MedioDePago
- ConcreteClass: TarjetaDeCredito, Transferencia y BilleteraVirtual

2.5 Notificaciones De Pedido

Patrón: Observer.

Justificación: Para la resolución de este punto utilizamos el patrón **Observer**. Ya que la constante evolución de Pedido debía de notificarse, nos encontramos con la necesidad de hacer una composición mediante el patrón Observer.

Roles de las clases en Composite:

- Subject: Pedido
- Observer: ObservadorPedido
- ConcreteObserver: NotificadorEmail, Fidelizacion y GeneradorFactura

2.6 Búsqueda en el catálogo**Patrón: Strategy y Composite.**

Justificación: Para la resolución de este punto utilizamos dos patrones juntos, **Strategy** y **Composite**. Implementamos un Strategy ya que el cliente que desee buscar va a poder modificar en ejecución la “estrategia” de búsqueda como guste. Encapsulamos la familia de algoritmos en un CriterioDeBusqueda. A su vez, al tener criterios que unen CriterioDeBusqueda o utilizar varios, nos encontramos con la necesidad de hacer una composición mediante el patrón Composite.

Roles de las clases en Strategy:

- Estrategia(Strategy): CriterioDeBusqueda
- ConcreteStrategy: CriterioPorNombre, CriterioPorCategoria, CriterioPorPrecioMaximo, CriterioPorDisponibilidad, CriterioAND, CriterioOR y CriterioNOT
- Context: Catalogo
-

Roles de las clases en Composite:

- Component: CriterioDeBusqueda
- Leaf : CriterioPorNombre, CriterioPorCategoria, CriterioPorPrecioMaximo y CriterioPorDisponibilidad
- Composite: CriterioAND, CriterioOR y CriterioNOT

2.8 Reportes**Patrón: Visitor.**

Justificación: Usamos el patrón Visitor para separar toda la lógica de exportación de lo que es el reporte en sí. La idea principal es que la clase ReporteMasVendidos solo se encargue de manejar los datos de los productos y no tenga que saber cómo formatearse a HTML o TXT. De esta forma, si el día de mañana nos piden exportar a otro tipo de formato, no tocamos para nada las clases de los reportes; simplemente creamos un nuevo visitante que implemente la interfaz ReporteVisitor.

Roles de las clases en Visitor:

- Visitor: ReporteVisitor

- Concrete Visitor: ExportadorTxt, ExportadorHtml y ExportadorCSV
- Element: Reporte
- Concrete Element: ReporteMasVendidos